CS 152 PROGRAMMING PARADIGMS

HOMEWORK 1
Due: Wednesday, 09/16/2026, by midnight on Canvas

Submission procedure: (a) Submit a report that contains both your code and printed outputs. (b) Submit all source files. No collaboration or use of AI is allowed for this assignment.

Question 1 **(10 points)**

Say you want to find a phone number in a string. You know the pattern: three numbers, a hyphen, three numbers, a hyphen, and four numbers. Here's an example: `000-000-0000`.

Let's use a function named `isPhoneNumber()` to check whether a string matches this pattern, returning either True or False. Open a new file editor window and save the file as isPhoneNumber.py.

Regular expressions, called regexes for short, are descriptions for a pattern of text. For example, a `\d` in a regex stands for a digit character—that is, any single numeral 0 to 9. The regex `\d\d\d-\d\d\d-\d\d\d\d` is used by Python to match the text containing phone numbers in the `isPhoneNumber()` function: a string of three numbers, a hyphen, three more numbers, another hyphen, and four numbers. Any other string would not match the `\d\d\d-\d\d\d-\d\d\d\d` regex. But regular expressions can be much more sophisticated. For example, adding a 3 in curly brackets `({3})` after a pattern is like saying, "Match this pattern three times." So the slightly shorter regex `\d{3}-\d{3}-\d{4}` also matches the correct phone number format.

All the regex functions in Python are in the **`re`** module. Enter the following into an interactive shell to import this module:

```
>>> import re
```

A Regex object's `search()` method searches the string it is passed for any matches to the regex. The `search()` method will return None if the regex pattern is not found in the string. If the pattern is found, the `search()` method returns a Match object. Match objects have a `group()` method that will return the actual matched text from the searched string.

Task: Use this information to write a python script to match phone numbers. Test your program on a sample search string containing a phone number. If you are comfortable using another language of your choice, please feel free to do so.

Question 2 **(10 points)**

Unix-based operating systems usually include a tool named tail. It displays the last 10 lines of a file whose name is provided as a command line argument.

Write a program that provides the same behavior i.e. the user can specify how many lines to print from the terminal. Display an appropriate error message if the file requested by the user does not exist, or if the command line argument is omitted.
Objective: To display the last n lines of a file whose name is provided as a command line argument.

Note: You may need to pass a file as an argument, so make sure to have access to a sample file, the contents of which you want to print out.

Extend this program to display the: `(a) first n lines of a file` and (b) `the middle n lines of a file`.

Question 3 **(15 points)**

Write a Java function called `EvenSquaresFilter`. The function takes two inputs: `an array of integers`, and an integer `threshold`. The function should:

(a) Find all even numbers in the array that are greater than threshold.
(b) Square each of those numbers.
(c) Remove any duplicate values from the resulting squares.
(d) Return the result sorted in descending order.

Then test your function in a main function using at least two different arrays and threshold values.

```
Original Array: [-6, -2, 0, 10, 10, 4]
Threshold: -5
Output: [100, 16, 4, 0]
```

If you are comfortable using another language of your choice for this problem, please feel free to do so.

Question 4 **(15 points)**

Write Verilog code to describe the following functions. You do not need to simulate using testbenches, code that compiles is sufficient. You can choose between `structural` or `behavioral` specifications.

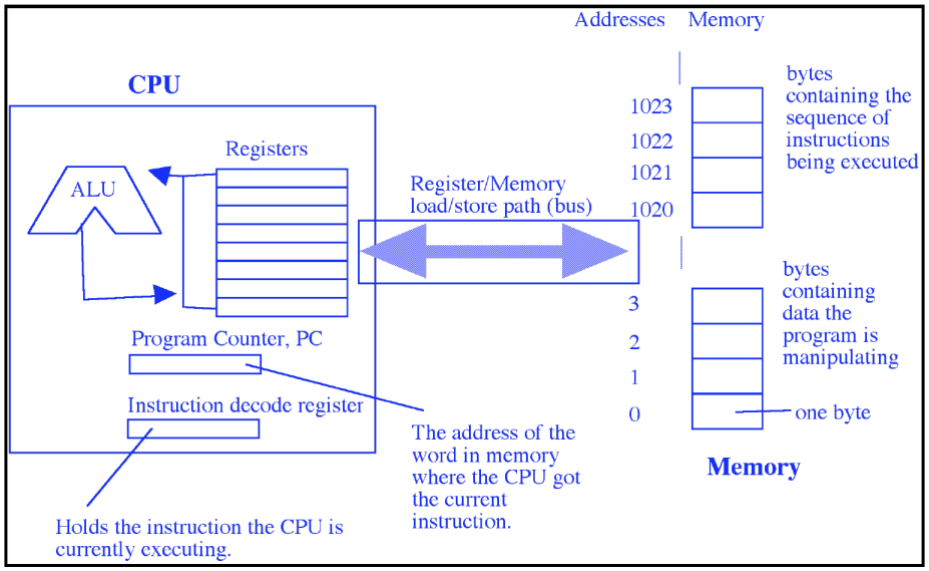
$$f_1 = x_1\overline{x_3} + x_2\overline{x_3} + \overline{x_3 x_4} + x_1 x_2 + x_1\overline{x_4}$$

$$f_2 = (x_1 + \overline{x_3})\,(x_1 + x_2 + \overline{x_4})(x_2 + \overline{x_3} + \overline{x_4})$$

Question 5* **(25 points)**

Complete the following case study below.

A simplified picture with the major features of a computer identified. The CPU is where all the work gets done, the memory is where all the code and data is stored. The path connecting the two is known as the "bus".



CPU

While memory stores the program and the data, the Central Processing Unit does all the work. The CPU has two parts— registers and an Arithmetic Logic Unit (ALU). A register is like the temporary memory in a calculator— each register can store a value that can be used in subsequent computations. Each register can usually hold one word. Sometimes there are separate specialized registers for holding specific types of data— floating point numbers, addresses, etc. The registers can be accessed much more quickly than memory, but the number of registers available is quite small compared to the size of memory. One of the specialized registers is the Program Counter, or PC, which holds the address of which instruction is currently being executed.

The ALU is the part of the CPU that performs the actual computations such as addition and multiplication along with comparison and other logical operations. Most modern, high-performance CPUs actually contain several specialized logic units in addition to the ALU which allow them to work on several computations in parallel. However, that complexity is kept (literally) within the CPU. The abstraction of the CPU is that it executes its instructions one at a time, in the order presented in memory.

For the balance of this exercise, we will concentrate on the instruction set of a typical Reduced Instruction Set Computer (RISC) processor. RISC processors are distinguished by a relatively lean instruction set. It's turned out that you can get the best overall performance by giving your processor a simple instruction set, and then concentrating on making the processor's instructions/second performance as high as possible. So RISC processors don't have some of the fancier instructions or addressing modes featured by older Complex Instruction Set (CISC) designs. Thankfully, RISC has the added benefit that it's easy to study since the instruction set is, well, reduced. Our fictitious processor has 32 registers, each of which can hold a 4-byte word. We will support three types of instructions: Load/Store instructions which move bytes back and forth between registers and memory, ALU instructions which operate on the registers, and Branch/Jump instructions that alter which instruction is executed next.

Load

Load instructions read bytes into a register. The source may be a constant value, another register, or a location in memory. In our simple language, a memory location is expressed Mem[address] where address may be a constant number, a register, or a register plus a constant offset. Load and Store normally move a whole word at a time starting at the given address. To move less than a whole word at a time, use the variants “=.1” (1 byte) and “=.2” (2 bytes).

Load the constant 23 into register 4
addi $r4, $zero, 5 (loading literal constant into register)

Copy the contents of register 2 into register 3
#add $r3, $r2, $zero

Load char (one byte) starting at memory address 244 into register 6
#lb $r6, 244($0)

Load R5 with the word whose memory address is in R1
R5 = Mem[R1]
lw $r5, 0($r1)

Load the word that begins 8 bytes after the address in R1. *This is known as "constant offset" mode and is about the fanciest addressing mode a RISC processor will support.*
#lw $r1, 8($r1)

Just to give a sense of how this relates to the "real world", the load instruction in Motorola 68000 assembly looks like this:

Move long (4 bytes) constant 15 to register d2
```
movel #15, d2
```

Move long at mem address 0x40c to register a0 `movel @#0x40c, a0`

Or in Sparc assembly, it looks like this:

Load from mem address at register o0 + constant offset 20 into register o1
```
ld [ %o0 + 20 ], %o1
```

The syntax of our assembly language is designed to make it easier to read and learn quickly, but the basic functionality is quite similar to any current RISC instruction set.

Store

Store instructions are basically the reverse of load instructions— they move values from registers back out to memory. There is no path in a RISC architecture to move bytes directly from one place in memory to somewhere else in memory. Instead, you need to use loads to get bytes into registers, and then stores to move them back to memory.

Store the constant number 37 into the word beginning at 400
#addi $t0, 37
#sw $t0, 400($s0)

Store the value in R6 into the word whose address is in R1
#sw $r1, $r6

Store lower half-word from R2 into 2 bytes starting at address 1024 Mem[1024] =.2 R2
sh $r2, 1024($0)

Store R7 into the word whose address is 12 more than the address in R1
Mem[R1+12] = R7
sw $r7, 12($1)

ALU

Arithmetic Logical Unit (ALU) instructions are much like the operation keys of a calculator. ALU operations only work with the registers or constants. Some processors don't even allow constants (i.e. you would need to load the constant into a register first).

Add 6 to R3 and store the result in R1
#li $r3, 6

add $r1, $r3, $0

Subtract R3 from R2 and store the result in R1 # sub $r1, $r2, $r3

Although we will use '+' for both indiscriminately, the processor usually has two different versions of the arithmetic operations, one for integers and one for floating point numbers, invoked by two different instructions, for example, the Sparc has add and fadd. Integer arithmetic is often much more efficient than floating point since the operations are simpler (e.g. require no normalization). Division is by far the most expensive of the arithmetic operations on either type and often is not a single instruction, but a small "micro-coded" routine (think of it as very fast hand-tuned function).

Just to give a sense of how this relates to the "real world", the load instruction in Motorola 68000 assembly looks like this:

Move long (4 bytes) constant 15 to register d2
```
movel #15, d2
```

Move long at mem address 0x40c to register a0
```
movel @#0x40c, a0
```

Or in Sparc assembly, it looks like this:

Load from mem address at register o0 + constant offset 20 into register o1
```
ld [ %o0 + 20 ], %o1
```

The syntax of our assembly language is designed to make it easier to read and learn quickly, but the basic functionality is quite similar to any current RISC instruction set.

Store

Store instructions are basically the reverse of load instructions— they move values from registers back out to memory. There is no path in a RISC architecture to move bytes directly from one place in memory to somewhere else in memory. Instead, you need to use loads to get bytes into registers, and then stores to move them back to memory.

Store the constant number 37 into the word beginning at 400
#addi $t0, 37
#sw $t0, 400($s0)

Store the value in R6 into the word whose address is in R1
#sw $r1, $r6

Store lower half-word from R2 into 2 bytes starting at address 1024
Mem[1024] =.2 R2
sh $r2, 1024($0)

Store R7 into the word whose address is 12 more than the address in R1
Mem[R1+12] = R7
sw $r7, 12($1)

ALU

Arithmetic Logical Unit (ALU) instructions are much like the operation keys of a calculator. ALU operations only work with the registers or constants. Some processors don't even allow constants (i.e. you would need to load the constant into a register first).

Add 6 to R3 and store the result in R1
#li $r3, 6

add $r1, $r3, $0

Subtract R3 from R2 and store the result in R1 # sub $r1, $r2, $r3

Although we will use '+' for both indiscriminately, the processor usually has two different versions of the arithmetic operations, one for integers and one for floating point numbers, invoked by two different instructions, for example, the Sparc has add and fadd. Integer arithmetic is often much more efficient than floating point since the operations are simpler (e.g. require no normalization). Division is by far the most expensive of the arithmetic operations on either type and often is not a single instruction, but a small "micro-coded" routine (think of it as very fast hand-tuned function).

Branching

By default, the CPU fetches and executes instructions from memory in order, working from low memory to high. Branch instructions alter this default order. Branch instructions test a condition and possibly change which instruction should be executed next by changing the value of the PC register. One condition that all processors make heavy use of is testing whether two values are equal or if a value is less (or greater) than some other value. The operands in the test of a branch statement must be in registers or constant values. Branches are used to implement control structures like if and switch as well as loops like for and while.

Begin executing at address 344 if R1 equals 0

```
# beq $r1, $0, 344     "branch if equal"
```

Begin executing at addr 8 past current instruction if R2 less than R3

BLT R2, R3, PC+8 "branch if less than"

The full set of branch variants:

BLT	branch if first argument is less than second
BLE	less than or equal
BGT	greater than
BGE	greater than or equal
BEQ	equal
BNE	not equal

Any branch instruction compares its first two arguments (which both must be registers or constants) and then potentially branches to the address given as the third argument.

The destination address can be specified as an absolute address, such as 356, or a PCrelative address, such as PC-8 or PC+12. The later allows you to skip over a few instructions or jump to a previous instruction, which are the most common patterns for loops and conditionals.

In addition, there is an unconditional jump that has no test, but just immediately diverts execution to a new address. Like branch, the address can be specified absolute or PCrelative.

Begin executing at address 2000 unconditionally- like a goto

Jmp 2000

Begin executing at address 12 before current instruction

beq $0, $0, -4 (jumping back to previous instructions)

Data conversion

Two additional instructions are utilities that convert values between integer and floating point formats. Remember a floating point 1.0 has a completely different arrangement of bits than the integer 1 and instructions are required to do those conversions. These instructions are also used to move values for computers that store floating point and integer values in different sets of registers.

For our purposes, we will have instructions that convert between the 4-byte integer and the 4-byte float value. The destination and source locations for the conversion operations must both be registers.

Take bits in R3 that represent integer, convert to float, store in R2

R2 = ItoF R3

Take bits in R4, convert from float to int, and store back in same. Note that converting in this direction loses information, the fractional component is truncated and lost
R4 = FtoI R4

Summary

Although there are a few things we bypassed (the logical and/or/not and some of the operations that support function call/return), this simple set of instructions gives you a pretty good idea of exactly what our average CPU can do in terms of instructions. The richness and complexity of a programming language like C is provided by the compiler which takes something complex like a for-loop, array reference, or function call and translates it into an appropriate sequence of the above simple instructions.

Question 6 **(25 points)**

Convert the following MIPS instructions to machine code:

```
(a)0x00600000:  lui   $s0, 0x7F3C
(b)0x00600004:  ori   $s0, $s0, 0x0021
(c)0x00600008:  addiu $s1, $s0, 5
(d)0x0060000C:  sra   $s2, $s1, 2
(e)0x00600010:  bgez  $s2, 0x00600028
```

a. lui (001111), rs (00000), rt (10000), 0x7F3C (011111110011110(
Machine Code: 0x3C107F3C

b. ori (001101), $s0 (10000), $s0 (10000), 0x0021 (0000000000100001)
Machine Code: 0x36100021

c. addiu (001001), $s1 (10000), $s0 (10001), 5 (0000000000000101)
Machine Code: 0x46110005

d. sra (000000), rs (00000), rt (10001), rd (10010), shamt (00010), funct (000011)
Machine Code: 0x00119083

e. opcode (000001), rs (10010), rt (00001), immediate (0000000000000101)
Machine Code: 0x06410005